

Artificial Intelligence Basics Course Summary

Generated by Scribe.AI

December 9, 2024

1 Key Terms

1.1 Fundamental Linear Algebra

- **Matrix**: A rectangular array of numbers, symbols, or expressions arranged in rows and columns.
- **Linear Transform**: A function that maps vectors from one vector space to another, satisfying $T(u+v) = T(u) + T(v)$ and $T(cu) = cT(u)$ for all vectors u, v and scalars c .
- **Matrix Multiplication**: An operation that combines two matrices to produce a third matrix, where the element at row i and column j of the resulting matrix is the dot product of row i of the first matrix and column j of the second matrix. The inner dimensions must match.
- **Matrix Inverse**: For a square matrix A , its inverse A^{-1} is a matrix such that $AA^{-1} = A^{-1}A = I$, where I is the identity matrix.
- **Matrix Transposition**: An operation that swaps the rows and columns of a matrix. The transpose of matrix A is denoted as A^T .
- **Derivative**: A measure of how a function changes in response to a small change in its input. It represents the instantaneous rate of change.
- **Partial Derivative**: A derivative of a function of multiple variables with respect to one of those variables, treating the others as constants.
- **Sigmoid Function**: A mathematical function that maps any input value to an output value between 0 and 1. Often used as an activation function in neural networks.

1.2 Artificial Neural Networks

- **Perceptron**: A fundamental unit in neural networks, consisting of weighted inputs, a summation node, and an output, used for binary classification.

1.3 Search Algorithms

- **Minimax Search**: A search algorithm for two-player zero-sum games that aims to find the best move for the current player, assuming the opponent also plays optimally.
- **Alpha-Beta Pruning**: A technique to reduce the search space in Minimax search by avoiding the evaluation of branches that cannot affect the optimal decision.
- **Game Tree**: A tree-like structure representing all possible game states and moves, with leaf nodes representing outcomes.

1.4 Reinforcement Learning

- **Reinforcement Learning**: A type of machine learning where an agent learns to make decisions by interacting with an environment and receiving rewards or penalties.

1.5 AI Foundations

- **Turing Test:** An operational test for intelligent behavior, the Imitation Game, where an interrogator tries to distinguish between a human and a machine based on their responses.
- **Inductive Reasoning:** The process of deriving general principles from specific observations.
- **Linear Regression:** A machine learning algorithm used to model the relationship between a dependent variable and one or more independent variables by fitting a linear equation to observed data.
- **Model Evaluation:** The process of assessing the performance of a machine learning model.
- **Overfitting:** A phenomenon in machine learning where a model learns the training data too well, resulting in poor generalization to unseen data.
- **Representative Data:** Data that accurately reflects the characteristics of the problem domain being addressed.
- **Binary Classification:** A machine learning task where the goal is to classify data into one of two categories.
- **Decision Boundary:** A line or surface that separates data points into different classes in a classification problem.
- **0-1 Loss:** A loss function in binary classification that assigns a loss of 1 if the prediction is incorrect and 0 if it is correct.
- **Perceptron Loss:** A loss function in binary classification that is 0 if the prediction is correct and $-z$ otherwise, where z is the margin.
- **Hinge Loss:** A loss function in binary classification that is 0 if the margin is greater than 1 and $1 - z$ otherwise, where z is the margin.
- **Logistic Loss:** A loss function in binary classification that is $\log(1 + e^{\sup>-z</sup>})$, where z is the margin.
- **Gradient Descent:** An iterative optimization algorithm used to find the minimum of a function by following the direction of the negative gradient.
- **Stochastic Gradient Descent:** A variant of gradient descent that updates the model parameters using a randomly selected subset of the data.

2 Problem Types

2.1 Foundational AI Concepts

- **Understanding the Nature of Intelligence:** This problem explores the definition and characteristics of intelligence, both human and artificial. It examines various perspectives on what constitutes intelligence and how it can be measured or simulated.
- **Defining Artificial Intelligence:** This problem involves formulating a precise definition of artificial intelligence, considering different perspectives and historical contexts. The challenge lies in capturing the essence of AI while accounting for its evolving nature and diverse applications.
- **Passing the Turing Test:** This problem focuses on the implications of passing the Turing Test, a benchmark for machine intelligence. It explores whether passing the test truly indicates machine intelligence or merely the ability to mimic human behavior.
- **Bridging the Gap Between Different AI Domains:** This problem investigates the similarities and differences between seemingly disparate AI domains, such as speech synthesis and game playing. It explores the potential for transferring knowledge and techniques across different AI applications.

- **Understanding the AI Grand Challenges:** This problem explores the reasons behind selecting specific problems, such as autonomous driving, as AI Grand Challenges. It examines the factors that make these problems particularly significant and challenging in the field of AI.

2.2 AI Algorithms & Mathematics

- **Overcoming the Limitations of Search:** This problem addresses the computational challenges associated with exhaustive search in complex games like chess and Go. It explores the need for alternative approaches, such as machine learning, to overcome the limitations of brute-force search.
- **Developing Robust and Generalizable Algorithms:** This problem focuses on the design of algorithms that can adapt to new situations and generalize their knowledge to unseen scenarios. It explores the challenges in creating algorithms that are not only effective in specific tasks but also robust and adaptable.
- **Representing Linear Transformations with Matrices:** Matrices are used to represent linear transformations, which map vectors from one space to another while preserving linear combinations. This is fundamental to many AI algorithms.
- **Matrix Arithmetic:** The slide covers basic matrix operations such as addition, subtraction, scalar multiplication, and multiplication, which are essential for many AI computations.
- **Solving Systems of Linear Equations:** Systems of linear equations can be represented and solved using matrices and their inverses. This is crucial for various AI applications.
- **Matrix Transposition:** Matrix transposition is a fundamental operation that involves swapping rows and columns of a matrix. It's used in various AI algorithms for data manipulation.
- **Parallelizing Matrix Multiplication:** Domain decomposition is presented as a method for parallelizing matrix multiplication, improving computational efficiency in AI.
- **Calculus in AI:** The slide briefly mentions the importance of calculus, specifically derivatives, partial derivatives, and finding maxima/minima/gradients, in AI algorithms.
- **Linear Regression:** Finding the line that best fits a set of data points, minimizing the sum of squared distances between the points and the line.
- **Multiple Linear Regression:** Extending linear regression to include multiple independent variables to predict a dependent variable.
- **Curve Fitting:** Fitting a curve to a set of data points, potentially using polynomial functions of higher degrees.
- **Binary Classification:** Classifying data points into two categories (+1 or -1) using a linear model and a suitable loss function.
- **Overfitting:** The phenomenon where a model fits the training data too closely, resulting in poor generalization to unseen data.
- **Choosing a Loss Function:** Selecting an appropriate loss function (e.g., 0-1 loss, perceptron loss, hinge loss, logistic loss) for binary classification, considering differentiability and computational efficiency.
- **Minimizing Loss using Gradient Descent:** Employing gradient descent or stochastic gradient descent to iteratively update model parameters and minimize the chosen loss function.

2.3 AI System Design & Implementation

- **Understanding the Architecture of Advanced AI Systems:** This problem explores the architecture and design of complex AI systems, such as AlphaGo, which combine deep learning and search techniques. It examines how these systems achieve superhuman performance in challenging tasks.

2.4 Specific AI Applications & Challenges

- **Addressing the Challenges of Computer Vision:** This problem investigates the difficulties in developing computer vision systems that can reliably interpret and understand images. It highlights the gap between initial expectations and the ongoing challenges in this field.
- **Mitigating AI Safety Risks:** This problem examines the potential risks associated with AI systems, such as adversarial attacks and biases. It explores methods for improving AI safety and ensuring responsible development and deployment.
- **Addressing Ethical Concerns in AI:** This problem focuses on the ethical implications of AI systems, particularly concerning bias and fairness. It explores the need for responsible AI development and deployment that considers societal impact and avoids perpetuating harmful biases.
- **Understanding the Differences Between Game Types:** This problem explores the differences between various game types, such as chess and soccer, in the context of AI. It examines how these differences affect the design and implementation of AI algorithms for playing these games.
- **Designing Effective Game-Playing Systems:** This problem investigates the fundamental components and strategies involved in designing AI systems capable of playing games effectively. It explores the interplay between learning, search, and strategic decision-making.

2.5 Neural Networks Basics

- **Understanding Perceptrons:** The perceptron, a fundamental building block of neural networks, is introduced. Its structure and function are explained using a diagram.
- **Derivative of the Sigmoid Function:** The slide shows the derivation of the derivative of the sigmoid function, a crucial step in training neural networks.

3 Algorithm Solutions

3.1 Linear Algebra

- **Matrix Addition:** $A + B = C$, where $c_{ij} = a_{ij} + b_{ij}$
- **Matrix Scalar Multiplication:** $k \cdot A = B$, where $b_{ij} = k \cdot a_{ij}$
- **Matrix Multiplication:** $C_{ij} = \sum_{k=1}^n a_{ik} b_{kj}$
- **Matrix Inverse:** $AA^{-1} = A^{-1}A = I$
- **Solving Linear Equations using Matrix Inverses:** $Ax = b \Rightarrow x = A^{-1}b$

3.2 Machine Learning Fundamentals

- **Multiple Linear Regression:** Find the optimal coefficients β that minimize the sum of squared errors between the observed and predicted values. This is done by solving the equation $\beta = (X^T X)^{-1} X^T y$, where y is the vector of observed values, X is the design matrix of independent variables, and β is the vector of coefficients.
- **Logistic Regression:** Use a linear model $w^T x$ and the sigmoid function $\sigma(z) = \frac{1}{1+e^{-z}}$ to predict the probability of a data point belonging to a class. The weight vector w is updated iteratively using stochastic gradient descent, with the update rule $w \leftarrow w + \alpha(\sigma(-y_n w^T x_n) y_n x_n)$, where α is the learning rate.

3.3 Artificial Neural Networks

- **Derivative of Sigmoid Function:** $\frac{d}{dx}\sigma(x) = \frac{e^{-x}}{(1+e^{-x})^2}$
- **Perceptron Algorithm:** Iteratively update the weight vector w by adding $y_n x_n$ if the data point x_n is misclassified (i.e., $\text{sign}(w^T x_n) \neq y_n$).

3.4 Game Playing AI

- **Minimax Search:** A recursive algorithm for two-player zero-sum games that explores the game tree to find the best move for the current player, assuming the opponent also plays optimally. It alternates between maximizing the score for the current player and minimizing the score for the opponent.
- **Alpha-Beta Pruning:** An optimization technique for Minimax search that reduces the search space by avoiding the evaluation of branches that cannot affect the optimal decision. It uses alpha and beta values to prune branches.